

W09D05 — Docker Image Size Optimization

ML Engineer Journey · Phase 3 — Docker & Clean Architecture · 8 May 2026

1. Kenapa Ukuran Image Itu Penting

Setiap kali image di-pull ke server, setiap kali container baru di-spawn untuk scaling, dan setiap kali CI/CD pipeline berjalan — kamu membayar biaya ukuran image tersebut. Image 1.2 GB yang di-pull 50 kali = 60 GB transfer. Image 200 MB di-pull 50 kali = 10 GB. Di production dengan auto-scaling, perbedaan ini nyata dalam waktu deploy dan biaya bandwidth.

Analogi: Image Docker yang besar seperti koper 30 kg untuk perjalanan 2 hari — overweight fee, nunggu bagasi, susah gerak. Production engineer tidak toleran dengan koper besar.

2. Base Image Variants Python

Variant	Ukuran	Isi	Rekomendasi
python:3.11	~1.01 GB	Full Debian + gcc, git, headers, semua ada	Development only
python:3.11-slim	~130 MB	Debian minimal, tanpa build tools	■ Production ML API
python:3.11-alpine	~52 MB	Alpine Linux, musl libc	Hati-hati — lihat catatan

■ Alpine **tidak selalu terbaik** untuk ML. musl libc sering incompatible dengan binary wheels numpy/scipy — pip terpaksa compile dari source, hasilnya lebih lambat dan kadang lebih besar. Untuk ML API, **python:3.11-slim** adalah pilihan terbaik.

3. Tiga Sumber 'Berat' di Image Python ML

- A. Base image yang salah** — python:3.11 bawaan sudah include gcc, git, curl, header files, dan puluhan package OS yang tidak pernah dipakai di runtime. Ganti ke slim = potong ~870 MB sekaligus.
- B. Cache pip** — setiap pip install secara default menyimpan .whl dan HTTP cache di /root/.cache/pip. Packages sudah ter-install, tapi cache-nya masih ada sebagai dead weight. Solusi: flag **--no-cache-dir**.
- C. Cache apt** — apt-get install menyimpan package list dan .deb files setelah install di /var/lib/apt/lists/. Juga dead weight. Solusi: hapus dalam RUN yang SAMA.

4. Teknik Optimasi — Ringkasan

Teknik	Implementasi	Dampak
Ganti base image	FROM python:3.11-slim	-870 MB
pip --no-cache-dir	pip install --no-cache-dir -r requirements.txt	-50-200 MB
apt cleanup 1 layer	apt-get install ... && rm -rf /var/lib/apt/lists/*	-20-60 MB
.dockerignore	Exclude venv/, .git/, data/, __pycache__/	-10-500 MB
Non-root user	adduser + USER appuser	0 MB (security)

5. Dockerfile Optimal (Single-Stage)

```

FROM python:3.11-slim

WORKDIR /app

# Kalau butuh OS package – install + cleanup dalam 1 RUN
RUN apt-get update && apt-get install -y --no-install-recommends \
    libgomp1 \
    && rm -rf /var/lib/apt/lists/*

# Copy requirements DULU – layer caching (W09D04)
COPY requirements.txt .

# --no-cache-dir = buang cache pip setelah install
RUN pip install --no-cache-dir -r requirements.txt

# Copy source code terakhir (paling sering berubah)
COPY . .

# Non-root user – security best practice
RUN adduser --disabled-password --gecos "" appuser
USER appuser

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

```

6. File .dockerignore

File .dockerignore bekerja seperti .gitignore — memberi tahu Docker apa yang TIDAK boleh di-copy saat instruksi COPY . . dieksekusi. Tanpa file ini, COPY . . akan membawa masuk seluruh isi project termasuk venv/ (ratusan MB), .git/, data/, dan __pycache__/.

```

.git/
__pycache__/
*.pyc
venv/
.venv/
*.ipynb
.ipynb_checkpoints/
data/
*.csv
*.xlsx
*.json
tests/
.pytest_cache/

```

```
.env
*.env
.vscode/
.DS_Store
```

■ Bukti nyata dari sesi ini: COPY . . menghasilkan layer 438 kB setelah .dockerignore diterapkan. Tanpa .dockerignore, layer yang sama bisa ratusan MB karena venv/ ikut ter-copy.

7. Cara Diagnosa Ukuran Image

Dua command utama untuk analisa layer:

```
# Lihat ukuran semua image
docker images | findstr nama-image

# Lihat breakdown per layer (dari atas = terbaru)
docker history nama-image:tag
```

Output docker history yang ideal untuk ML API (hasil sesi ini):

LAYER	SIZE
CMD	0B
EXPOSE	0B
USER appuser	0B
adduser appuser	73.7 kB
COPY . . ■	438 kB ← .dockerignore bekerja
pip install ■	390 MB ← numpy/sklearn/fastapi, wajar
COPY requirements.txt	12.3 kB
WORKDIR /app	8.19 kB
python:3.11-slim base	~140 MB

8. Batas Single-Stage Build

Setelah semua teknik hari ini diterapkan, sisa berat image (~390 MB) berasal dari numpy, scikit-learn, pandas, fastapi, dan dependencies-nya. Package-package ini memang berat dan tidak bisa dihindari di single-stage build.

Satu-satunya cara mengurangi lebih jauh: Multi-Stage Build (W12D01). Dalam multi-stage build, stage pertama melakukan compile dan install semua dependencies, kemudian hanya binary/hasil akhir yang di-copy ke stage kedua (production image). Image final tidak membawa source code, cache build, compiler, atau build tools — secara teori menghasilkan image yang jauh lebih kecil.

	Single-Stage (hari ini)	Multi-Stage (W12D01)
Kompleksitas Dockerfile	Sederhana	Lebih kompleks (2+ FROM)
Ukuran image	~200–700 MB	Bisa jauh lebih kecil
Build time	Lebih cepat	Lebih lambat (2 stage)
Cocok untuk	ML API dengan banyak packages	Go/Rust/compiled apps

